Flowchart

Binary Tree Node Counting (Modified Version) : four flowcharts with explanations

1. Introduction to the Flowcharts

The flowchart represents the modified binary tree node-counting program, including counting nodes with exactly two children, displaying leaf node values, and comparing the recursive and iterative methods. Four flowcharts are given: **A** the main program (comparison), **B** counting the different types of nodes, **C** the recursive total count, and **D** the iterative total count using a queue. Each is followed by a block-by-block explanation.

2. Flowchart Symbols Used

Symbol	Name	Meaning	Used for
Rounded oval	Terminal	Start, end or return point	Start, Stop, End, Return
Rectangle (green)	Process	An operation or calculation	totalNodes++, leafNodes++, twoChildNodes++, count++ ...
Diamond	Decision	A Yes / No question	Node is NULL?, Leaf node?, Both children exist?, Queue empty?
Rectangle (purple)	Call / queue operation	A recursive call or a queue push	count(left); count(right), Push left child
Rounded box (red-tinted)	Return value / warning	A value returned to the caller or an error path	Return 0, Return count, Check the algorithm
Arrow	Flow line	Direction of control	Connects all blocks

3. Rules Followed While Drawing

- Every flowchart has one start and at least one end or return.
- Flow goes from top to bottom; branches go to the right and rejoin the main line.
- Each diamond has one entry and two labelled exits (Yes / No).
- Arrows do not pass through other blocks; loops use a bent return arrow.
- Branches that lead to the same next step are merged before continuing.

Note: the traversal and counting decisions are performed for each non-NULL node. Recursive calls return when they reach NULL children. The final counts are displayed only after the initial traversal finishes.

4. Flowchart A : Comparing Recursive and Iterative Methods (Main Program)

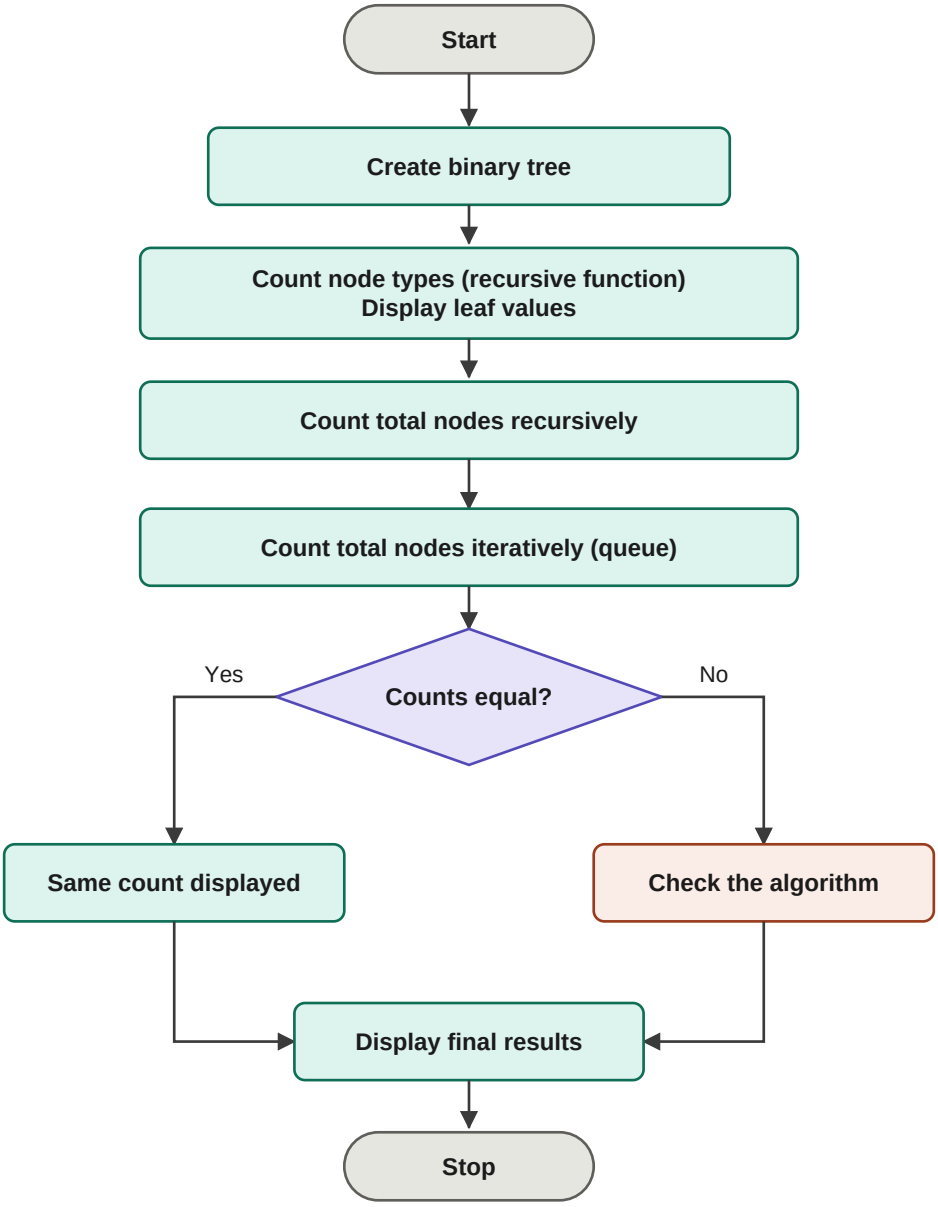

Figure 1: Flowchart of the main program

Block	What it does
Create binary tree	Allocates the nodes and links them through left and right pointers.
Count node types, display leaf values	Runs Flowchart B; prints the leaf values and fills the five counters.
Count total nodes recursively	Runs Flowchart C and stores the returned value.
Count total nodes iteratively (queue)	Runs Flowchart D and stores the returned value.
Counts equal?	Compares the two totals. Yes: both methods agree. No: there is an error in one of the algorithms.
Display final results	Prints all counts, both totals and the comparison message.

5. Flowchart B : Counting Different Types of Nodes

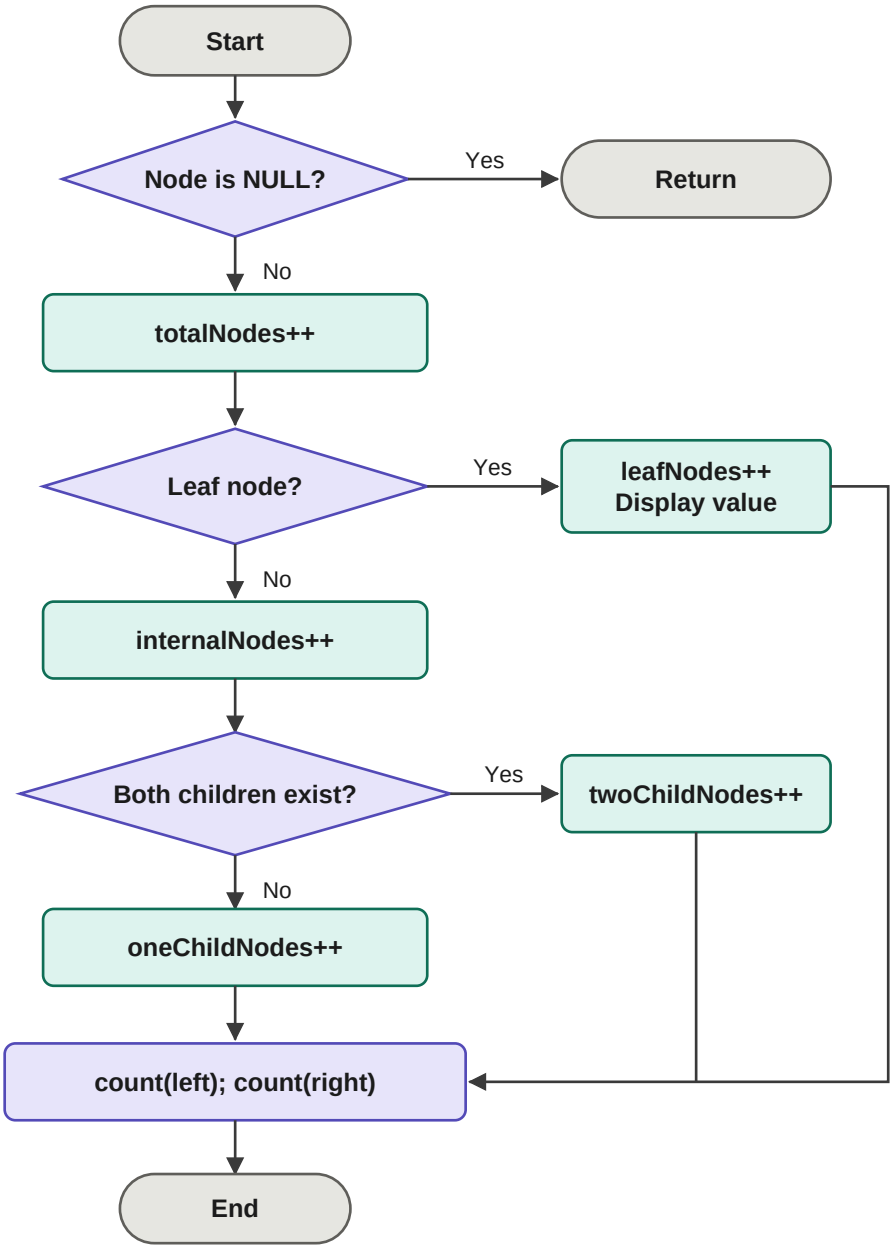

Figure 2: Flowchart of the recursive function that counts node types and displays leaf values

6. Explanation of Flowchart B

Block	Type	Explanation
Start	Terminal	A call of the function begins.
Node is NULL?	Decision	Base case. Yes: the subtree is empty, so return. No: continue.
Return	Terminal	Ends this call and goes back to the caller.
totalNodes++	Process	Counts the current node (executed for every non-NULL node).
Leaf node?	Decision	Yes if both child pointers are NULL.
leafNodes++ / Display value	Process	Increases the leaf counter and prints the data of the leaf. Then joins the recursive call block.
internalNodes++	Process	Reached on the No path: the node has at least one child.
Both children exist?	Decision	Yes: two children. No: exactly one child (the leaf case is already removed).
twoChildNodes++	Process	Counts a node with two children, then joins the recursive call block.
oneChildNodes++	Process	Counts a node with exactly one child.
count(left); count(right)	Call	The same flowchart is run for the left and then the right subtree.
End	Terminal	The call finishes and control returns to the caller.

7. Decision Logic Table

left	right	Path through Flowchart B	Counters changed
NULL	NULL	Not NULL -> total -> Leaf? Yes -> leaf, display value	total, leaf
exists	NULL	Not NULL -> total -> Leaf? No -> internal -> Both? No -> one	total, internal, one
NULL	exists	Not NULL -> total -> Leaf? No -> internal -> Both? No -> one	total, internal, one
exists	exists	Not NULL -> total -> Leaf? No -> internal -> Both? Yes -> two	total, internal, two

8. Walkthrough of Flowchart B on an Example Tree

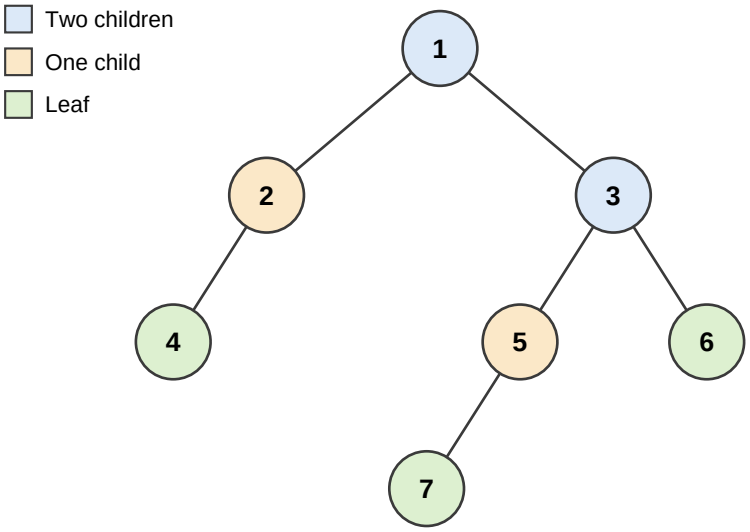

Figure 3: Example binary tree

Example tree used in all documents: node 1 is the root with children 2 and 3; node 2 has only a left child (4); node 3 has children 5 and 6; node 5 has only a left child (7).

Node	Path through Flowchart B	Result
1	NULL? No -> total++ -> Leaf? No -> internal++ -> Both exist? Yes -> twoChild++ -> count(2), count(3)	Two children
2	NULL? No -> total++ -> Leaf? No -> internal++ -> Both exist? No -> oneChild++ -> count(4), count(NULL)	One child
4	NULL? No -> total++ -> Leaf? Yes -> leaf++, display 4 -> two NULL calls	Leaf
3	NULL? No -> total++ -> Leaf? No -> internal++ -> Both exist? Yes -> twoChild++ -> count(5), count(6)	Two children
5	NULL? No -> total++ -> Leaf? No -> internal++ -> Both exist? No -> oneChild++ -> count(7), count(NULL)	One child
7	NULL? No -> total++ -> Leaf? Yes -> leaf++, display 7 -> two NULL calls	Leaf
6	NULL? No -> total++ -> Leaf? Yes -> leaf++, display 6 -> two NULL calls	Leaf

Result: leaf values shown: 4 7 6; total = 7, leaf = 3, internal = 4, one child = 2, two children = 2.

9. Flowchart C : Recursive Node Count

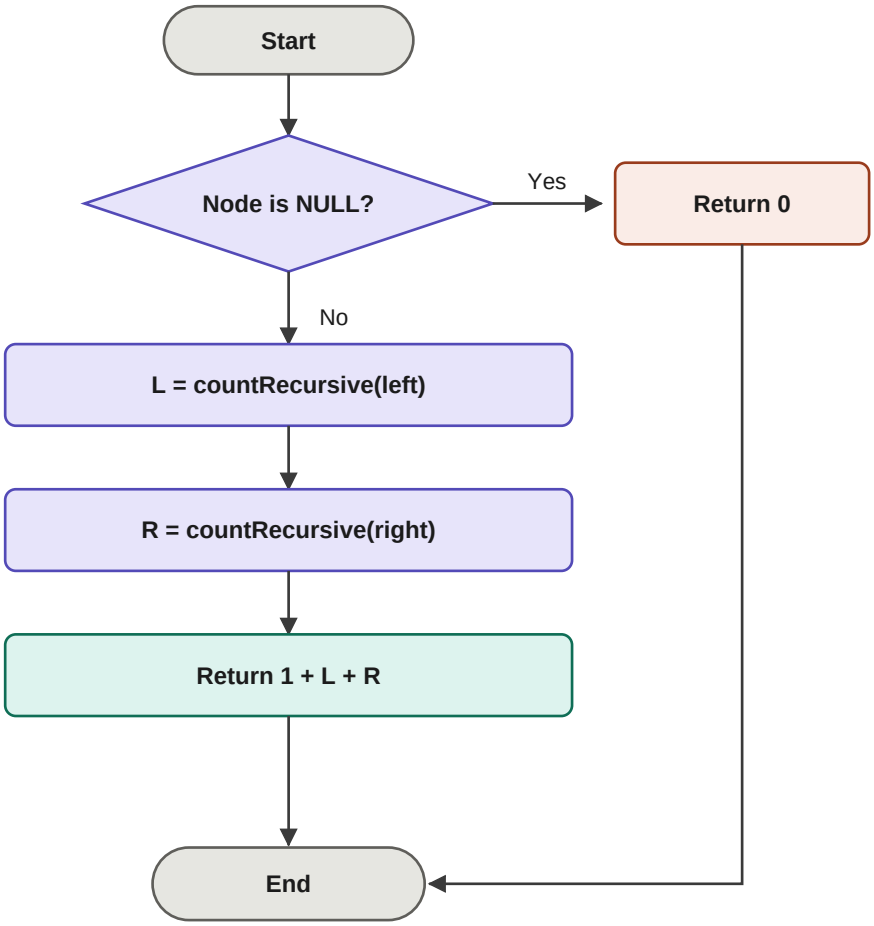

Figure 4: Flowchart of countRecursive(node)

Block	Explanation
Node is NULL?	Base case. An empty tree has zero nodes.
Return 0	Returned to the caller when the subtree is empty.
L = countRecursive(left)	Calls the same function for the left subtree and stores its node count.
R = countRecursive(right)	Calls the same function for the right subtree and stores its node count.
Return 1 + L + R	Formula $N = 1 + N_L + N_R$: the current node plus the nodes of both subtrees.

Recursion is not drawn as a separate copy for every call. The two call blocks mean 'run this same flowchart again for that child and wait for the answer'. The Yes exit of the NULL test stops the repeated calls.

10. Flowchart D : Iterative Node Count Using a Queue

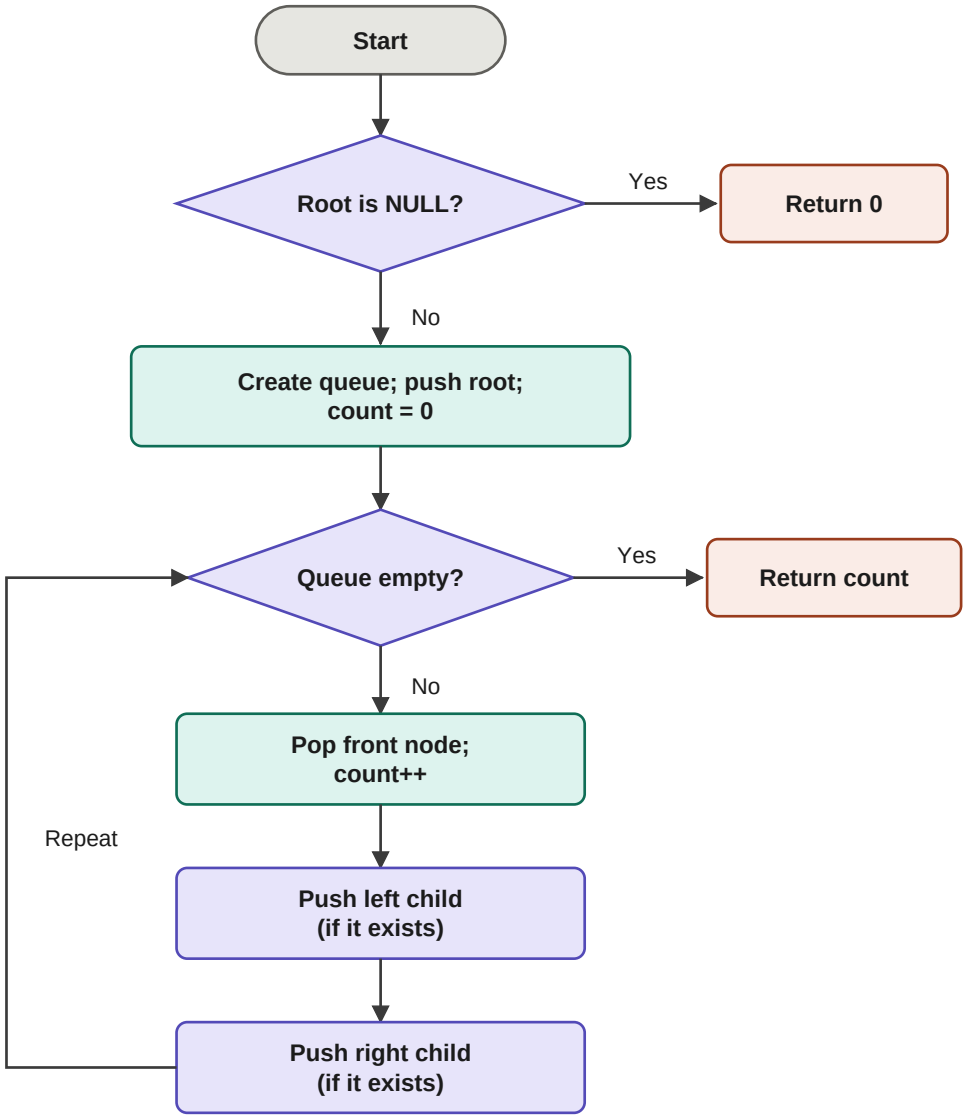

Figure 5: Flowchart of countIterative(root)

Block	Explanation
Root is NULL?	An empty tree has no nodes, so return 0 immediately.
Create queue; push root; count = 0	The queue holds nodes waiting to be visited; the root is the first one.
Queue empty?	Loop condition. Yes: every node has been counted, so return count. No: process the next node.
Pop front node; count++	Removes the oldest waiting node and counts it.
Push left child (if it exists)	Adds the left child to the queue so that it is visited later.
Push right child (if it exists)	Adds the right child. The arrow then returns to 'Queue empty?'.

11. Comparison of Flowcharts C and D

Point	Flowchart C (recursive)	Flowchart D (iterative)
Repetition done by	The function calling itself	A loop that runs until the queue is empty
Stop condition	Node is NULL	Queue is empty
Memory used	Call stack, O(h)	Queue, O(w)
Shape of the diagram	Straight flow with two call blocks	Contains a loop-back arrow
Result for example tree	7	7

12. Common Flowchart Mistakes

- Missing the NULL check, so the recursion never stops.
- Drawing the one-child and two-child tests in a way that lets one node pass through both counters.
- A decision diamond such as 'Has any child?' drawn with no Yes / No exits.
- Forgetting to merge the leaf, one-child and two-child branches before the recursive call block.
- Leaving out the loop-back arrow in the iterative flowchart.
- Using a rectangle for a question (it must be a diamond).

13. Summary

Flowchart B decides the type of each node with three diamonds (NULL, leaf, both children) and a recursive call block. Flowcharts C and D count the same total in two different ways, and Flowchart A compares their answers. All four flowcharts together describe the complete modified program.